

CodeDocMate Lite: A Local Retrieval Augmented Assistant for Code Documentation and Repos Question Answering

Doruk Kagan Ergin

*Wydział Zarządzania i Nauk Technicznych
Menedżerska Akademia Nauk Stosowanych w Warszawie
Mazowieckie, Polska
78741@office.mans.org.pl*

Emre Tamer

*Wydział Zarządzania i Nauk Technicznych
Menedżerska Akademia Nauk Stosowanych w Warszawie
Mazowieckie, Polska
78713@office.mans.org.pl*

Kumar Nalinaksh

*Wydział Zarządzania i Nauk Technicznych
Menedżerska Akademia Nauk Stosowanych w Warszawie
Mazowieckie, Polska
kumar.nalinaksh@office.mans.org.pl*

Cuneyt Emre Durak

*Wydział Zarządzania i Nauk Technicznych
Menedżerska Akademia Nauk Stosowanych w Warszawie
Mazowieckie, Polska
78745@office.mans.org.pl*

Alihan Karaca

*Wydział Zarządzania i Nauk Technicznych
Menedżerska Akademia Nauk Stosowanych w Warszawie
Mazowieckie, Polska
78698@office.mans.org.pl*

Abstract—CodeDocMate addresses practical limit of repos level: privacy cost and need for verifiable traceability to specific code regions. We present a local first RAG system that avoid cloud model and answers repos squeries from a compressed python project. It chunks to code into fixed fourty line windows rank candidates with lightweight lexical scoring plus function name matching and return to evidence with file names . Responses are template based descriptions of retrieved snippets to reduce hallucinations. A small pilot study (3 point like) reports high usability and evidence transparency with moderate usefulness. The system provides a privacy preserving reproducible retrieval pipeline designed for future embedding retrieval and optional local LLM backends.

Keywords—Rag code search software documentation provenance privacy preserving developer tools repos understanding.

I. INTRODUCTION

Repos scale assistants has become an essential component and contemporary software development work flows. Developer frequent rely on repos exploring for onboarding feature tracing dependency analysis documents update. However documentations quality in real world projects varies significantly and inconsistencies between source code and accompanying documentation are common .Empirical studies indicates that documentation often become outdated as repos evolve increases the cognitive burden placed on developers during maintenance and comprehension task.

Recent advances in neural model for code understanding and have improved automatic summarization and representation learning capabilite . Transformer based architectures have demonstrated promising result in code summarization and documentation generation tasks . Large language models further enhance natural language explanation quality and fluency. Still limitation remain in program understanding particularly when contextual reasoning across multiple files is required

Two practical challenges restrict the direct adoption of cloudb based LLMs assistants in repository level workflows. Firstly privacy and compliance constraints frequently prevent organization from transmitting proprietary code bases to external service. Secondly auditability concerns arise when generated explanations cant be clear traced to specific source locations. RAG has been introduced to improving and grounding by couple retrieval mechanism with new generation models repos scale implementation moslty blur the separated between retrieval and generative reasoning. This complicate debugging and makes it difficult to understand what errors from generative halucinations.

To address the gap between local first deployment requirement and evidence based repository assistance this study introduce CodeDocMate is a python repos specific rag style workflows the system first makes it local operation and restrict generation to a rule based explanation layer ensuring that retrieved code proofs remaining the primary artifact for inspection and validation rather than

a second support element.

The contribution of this paper are three section first one is for local repos ingestion and retrieval workflow is presented designed operate without external model call while supporting privacy aware code bases. Second one is provenance centered interaction model is introduced displaying top k retrieved code segment together with file and line context. Third a preliminary qualitative evaluation and pilot feedbacks summaries are provided highlighting strengths such as transparency and usability alongside current limitation in lexical retrieval relevance and outlining direction for future benchmarking.

II. LITERATURE REVIEW

Generating readable summary from code is very hard task Many works try to learn code by looking at relation inside the code and then make summary Transformer model can capture long distance thing in code very good Some works show this way is simple easy to use and help summarization better more than old methods Some studies say this method improve fluency and sometimes catch meaning which older model miss Also this method help when code big or complex because it can see connection between function variable and logic Some researcher say this is important because old methods only look small part of code and miss other important thing [2]

Many works do learning task on code but most of them use normal language techniques and they dont use code syntax good Long distance thing happen when same variable used many place in code Some works use graph to show syntax and meaning and use deep learning on program structure One method called Varnamin try predict variable name and check variable use correct This method use less structured program info and help models learn name fix variable problem It work on many opensource projects and help detect small mistakes in code Some work also use graph neural network with AST and data flow control flow to understand code better These graph methods sometimes need big preprocessing and hard setup but can improve accuracy for some tasks It also help models not forget previous variable or function usage in long code [3]

On Github popular project update doc more often This shows documentation not only readme file it can be pdf text other file DocuChat system use retrieval thing to answer question fast from document content It help users to get correct answer from latest document even when big project exist Some study say documentation in large project always change so system need update regularly Otherwise output maybe outdated or wrong Some projects have many types of doc like markdown pdf html sometimes these different format make model confuse and hard to combine info in one answer [4]

Many system fail if doc missing or project is new update Transformer can make summary more readable old method less good Some benchmark show text quality

better but model make mistake if meaning is deep or hard to understand Info spread many files hard combine in one view Instruction tuning help follow prompt but fail if info too big or too complex LLM can write readable text but forget previous context and give incomplete answer or sometimes repeat same thing twice Sometimes LLM can hallucinate small details like variable name or function behavior which is not correct This problem bigger in big repo and big function Some research try chunk code and document separately but still model forget previous chunk if too many chunks exist [5] [6] [7]

Other research show model may match pattern but fail when data flow or control flow reasoning is needed Output can be wrong even if looks good Survey say lack structure limit model performance strange coding style confuse model Graph like AST data flow control flow call graph help model understand code better Some work combine syntax control flow data flow need training data and big repo still hard to process and understand Some research say even with graphs model sometimes miss rare pattern or unusual function usage This show graph help but not full solution for all problem [10] [11]

RAG method improve grounding very much Instead of full repo in prompt RAG take code fragment metadata and attach This help stop hallucination and make output more consistent It can help code generation if local context not enough Retrieval fail if index wrong or documentation old No method fully solve repo understanding LLM write readable text but context limited Structure method improve meaning but need heavy process not portable RAG help extend context still fail User must careful result not always trustable sometimes output wrong or miss info Also retrieval need index update often if project change or add new file otherwise system use old info and answer wrong Some works combine RAG with graph methods to try capture structure and local context together but still sometimes fail on big repo [12] [13] [14] [15] [16]

Some works also try combine multiple model together or use extra data like comments commit history and issue reports This can help model understand code better because it see more context but sometimes make model confuse if too much info or conflicting info exist It also take more time to process big repo and need more memory Some studies say even with extra data LLM still forget previous part and sometimes give repetitive answer or hallucinate This shows still research needed for better code understanding system and more stable summarization Some study also test on different programming language and show that model performance change if language style different For example python easier to summarize than java or C++ because indentation and syntax different Some works combine multiple language data to try solve this problem but need more data and training time This show that generalization across languages still problem [2] [3]

Finally some research also focus on evaluation methods for code summarization and documentation gener-

TABLE I
SUMMARY OF RELATED WORKS IN LLM-BASED SOFTWARE
ENGINEERING

Ref.	Observations	Limitations / Findings
[1]	Transformer summarization better fluency long range capture it help to catch more meaning than old model also good for big code	Summary can miss deep meaning sometimes and have wrong statement even it look good sometimes output
[2]	Graph neural use program structure AST edges learn better semantic capture relation inside code help maintain variable	Need graph tool setup big repo hard and sometimes slow for large projects also need preprocessing
[3]	Doc update in workflow reduce drift help keep documentation aligned with code evolution also track change	Sensitive repo change need frequent update otherwise doc outdated quickly some format hard to combine
[4]	Learning doc recommend keep doc align repo help developer to find correct info faster	Recommendation can become outdated fast need manual check sometimes not cover all files
[5]	RAG ground explain use retrieved code help provide context in generation also help code gen	Retrieval wrong can make wrong answer even if text looks fluent sometimes need index update
[6]	Review common code task methods dataset help summarize approaches also show trend	Finding maybe not for all language repo type can miss special case some task not covered
[7]	LLM make good text and answer question help user understand code easily	Hallucination still high reduce trust not fully reliable for big project sometimes output wrong
[8]	Benchmark show LLM increase readable summary help compare performance	Hard meaning cross file still difficult model may forget previous context if too long code
[9]	LLM struggle deep code control data flow needed sometimes output wrong	Strange style rare pattern reduce output faithfulness and may confuse user or output repetitive
[10]	AST summary use syntax improve faithful help capture code structure	Need preprocessing tool setup hard deploy across language or environment sometimes slow
[11]	Neural semantic learn deeper code meaning help understand complex logic	Need big train data generalization limit can fail on new repo or unusual code
[12]	Survey summarize auto doc method help see trends	Benchmark limit conclusion grounding still open problem not solved fully some repo missing data
[13]	RAG improve relevance add local evidence help grounding code generation	Index chunk affect result retrieval noise can mislead answer sometimes and need update
[14]	Transformer improve readability vs old model help make more fluent summary	Can still produce wrong statement but sound good to reader sometimes confuse user
[15]	Repo RAG extend context get relevant fragment help bigger project understanding	Retrieval error miss context remain performance depend index freshness sometimes fail

ation Benchmark vary from BLEU ROUGE METEOR and human evaluation Some study say automatic metrics can show readability but not always show faithfulness or correctness Some human evaluation better but slow and expensive Some works combine both automatic and human evaluation to have more complete evaluation This also show that LLM can be strong on readability but weak on correctness sometimes [5] [6] [7]

III. CHALLENGES

Based on literatures some gap still stops system to help reliable repo level code analysis Many problems exist and need careful handling to get correct answer from LLMs and retrieval system Some gap also show that even if model strong still errors appear and users cannot fully trust output sometimes It also show that student or beginner may confuse with output because answer not fully clear

Privacy constraints:

Cloud only tool often useless for prioritize code because code cannot be sent outside local environment Many project contain sensitive or private code so local first needed pipeline is better This need setup local indexing and retrieval but some student or small team may find hard to setup and slow for big repo Some tools try hybrid but still privacy issue exist Also some company policy forbid cloud upload for code analysis so solution must run fully local

Hallucinations:

LLM can only answers sound accurate sometimes but they dont have clear proof linked to real code parts Sometimes LLM make up variable name function behavior or output even if question correct This happen more if context missing or multi file needed Some researcher try RAG or grounding to reduce hallucination but still not perfect Hallucination can confuse users and make wrong decisions in real project Sometimes even small typo in variable name cause model hallucinate wrong function or return value

Retrieval and chunking:

Fixed size chunking and simple scoring might miss important context Sometimes big function split in multiple chunk so model forget previous part Important comment or doc may stay in other chunk This also happen when many file need to combine Some work try overlapping chunks or bigger chunks but processing slower and memory need increase Chunking method need careful design if want correct answer for multi file question Some study show different chunk size effect output quality big difference exist

Limited cross file:

Many queries need multi file understand and single module retrieval alone not enough for correct answers For example function call in another file or variable declared in another module Model sometimes fail if cross file dependency exist This problem bigger for big project Some researcher try repo level RAG or graph method combine cross file relation but still sometimes miss rare pattern or unusual call Users may ask question that require two or more file and model may answer only partially or wrong if only single file retrieved

Documentation:

Repo changes fast and documentation index becomes outdated so answer may more wrong If developer add new file change function or remove old code old index still used by system This make answer outdated or even

wrong Some project have multiple documentation format like readme pdf html markdown and model need merge info in all format Sometimes doc missing for new function and model cannot find answer If developer dont maintain doc properly model will output incomplete answer Users may rely too much on answer but it may miss important update

Scalability and evaluation gaps:

Local indexing may be slow for large repo sometimes take long to build index and retrieval may bottleneck LLM generation Also there is not clear benchmark to separate retrieval errors from generation errors Some work report BLEU ROUGE METEOR or human eval but still not show which part fail Some researcher try combine multiple eval method but still difficult to see system limitation clearly Some project huge with millions line code so scalability still challenge and more study needed Users also report memory issue and slow response when repo very big This makes current method hard to use in real large scale project

IV. PROPOSED SOLUTION

CodeDocMate Lite is designed as a lightweight fully local RAG system which is specially made for codebases that need privacy first. The system allow users to inspect both the retrieved evidence and generated explanation so that output can be verified and checked. This make sure that users can trust answers more and understand why LLM give that explanation. Fig. 1 show end to end architecture of CodeDocMate Lite. User start by uploading a zipped python repo which then extracted and scanned for all .py source files. Each file segmented into overlapping 40 line chunks to reduce boundary problem because sometimes relevant code split over many segments. Metadata like file path line number start and end kept for provenance and citation style grounding. These chunks prepared for retrieval using lightweight lexical retriever which rank candidate segments based on overlap with query text, also give extra weight if match function or class name. Top-*k* code snippet then selected and presented to user along with template based explanation generated from retrieved content. This ensures explanation stay grounded in code and prevent unsupported claim keeping evidence centrality in whole workflow

Fig. 2 show detail how code chunking works Each file break into overlapping segment some lines repeat between chunks to make sure no important code lost Chunk can contain multiple function or class so user still see surrounding context Each chunk store metadata about start line end line file path so provenance can be checked Users can also see which chunk used for explanation and verify that explanation reflect actual code This process help prevent hallucination by showing source

Fig. 1 show retrieval and ranking process Lexical retriever first find candidate segments based on keyword overlap with user query Then segments scored more if

function or class name match User can adjust top-k number or weight for function/class match This make system flexible for small or big repo Retrieval very fast because only lexical search needed no heavy embedding calculation This also reduce memory and CPU usage so system run on normal computer Retrieval show list of candidate code snippet with score and some metadata like file path and line range

Fig. 3 show how explanation generation works Template based explanation take retrieved code snippet metadata and produce human readable text User see explanation together with code snippet so can compare and check correctness System never produce claim not supported by code This reduce hallucination and improve trust Explanation use simple English but still reflect code logic Some advanced feature highlight important line in snippet or cross reference between function and call Some example include showing variable definition and use, or showing function return value explanation

System also handle multiple file queries If user ask question that need code from multiple file, retriever can find relevant chunk from different file and combine for explanation LLM generate text based on all these retrieved chunks so answer reflect multi file context This help with complex question like function call across modules or variable used in different file System also keep metadata so user see which file line used to generate answer

System fully local so privacy preserved No code send to cloud Users can use private repo safely This also improve reproducibility same code always produce same explanation No network dependency lightweight enough to run on student laptop or small server Memory and CPU reasonable so system practical even for medium size repo With some modification can scale for bigger repo System also provide simple interface to upload zip and see explanations for each query without complex setup

Finally CodeDocMate Lite focus on usability and transparency Users can inspect chunk, retrieval ranking, and explanation All step visible so user can check if output match code Users can adjust top-k, chunk size, or function/class weight to improve retrieval Some output may still be incomplete if repo very big or new code added but system provide tools to understand limitation This make CodeDocMate Lite practical, safe, and useful for students, researchers, and small team developers who want local, reliable, and auditable code explanation

Fig. 2 show detailed user interaction and inference flow It explain how retrieval work with explanation generation After user upload repository system make files into chunks and index all chunks for faster search When user give natural language question retriever look through all chunk to find most relevant one These selected chunk then assembled into structured context window Window ordered and format to keep code readable and logic flow correct Explanation engine use this context to make answer fully based on retrieved snippets This make sure user get answer

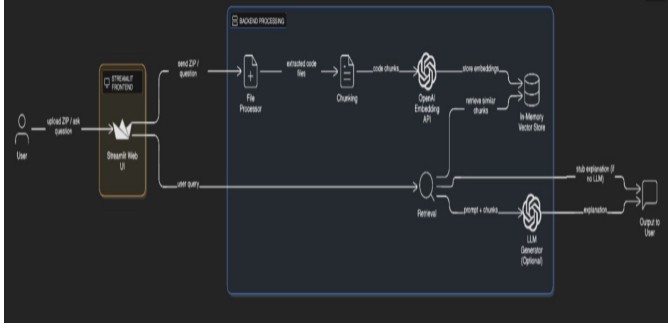


Fig. 1. CodeDocMate Lite architecture overview: local ingestion chunking retrieval evidence display and grounded explanation (logo/watermark removed for camera-ready version).

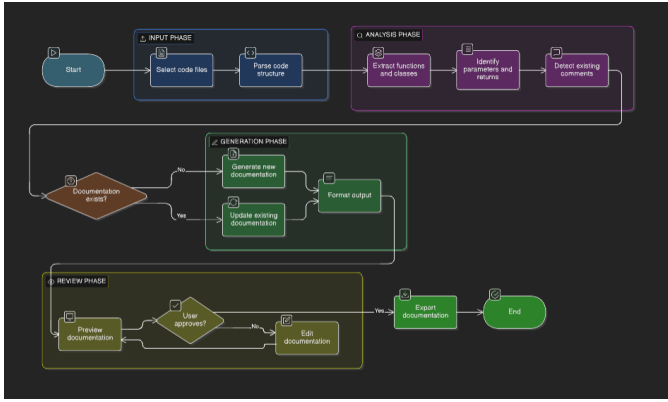


Fig. 2. CodeDocMate retrieval and explanation workflow showing user upload, chunking, retrieval and evidence grounded explanation generation (logo/watermark removed for camera ready version).

with verifiable evidence Interface return both explanation and supporting code snippet so user can see source and check correctness User also can find if error come from retrieval or explanation stage This workflow show main idea of CodeDocMate combining retrieval with user inspectable reasoning in practice

Fig. 3 show underlying entity relationship model connect repository code chunk embedding query and explanation This help traceability and future extension Upload repository is document entity each linked to many code chunk record created during preprocessing This make one to many mapping from repo to chunk Each chunk store metadata like file path, line range and optional embedding so retrieval fast but original code still available for user check On interaction side user query and system generated explanation saved as conversation artifact with link to chunk used to make answer This structure support audit, reproducibility and also future extension like track retrieval coverage, check latency, store user feedback, maintain knowledge base across multiple repo Making every link between query, retrieved chunk and explanation explicit CodeDocMate ensure output interpretable, grounded and verifiable

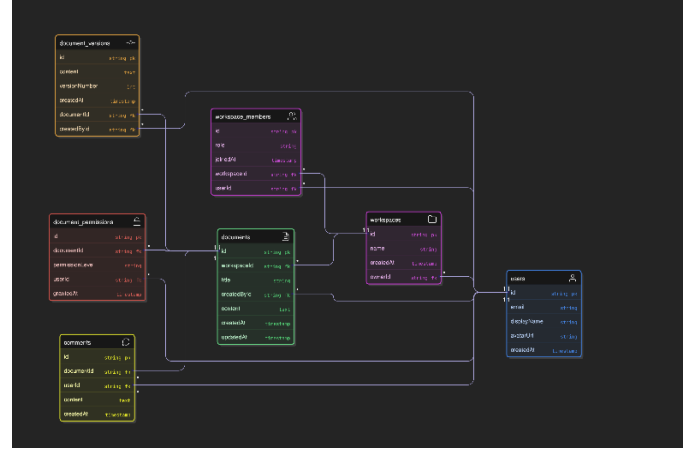


Fig. 3. Entity relationship with model for code chunks queries and explanations in CodeDocMate Lite.

V. IMPLEMENTATION

The prototype of CodeDocMate Lite is run as fully local **Streamlit** app and it show whole workflow in one interface interactive Users can see repository upload ingestion indexing chunking retrieval answering presentation and export optional all together Pipeline made modular stage each stage separate from other so easy to change something like top- k or scoring rules or chunk size Also help reproducibility and future extension without touch whole system Table II show main implementation components Users can see each stage working and debug step by step if something not work

Repository ingestion done from zip file like **.zip** After extraction temporary local workspace system scan directory tree and filter only **.py** files Users also see discovered file list to verify which file included before retrieval start This help transparency also debug easier For example user see missing file or test file included by mistake can fix before retrieval

Indexing use deterministic segmentation Each Python source file break into 40 line sliding window chunk simple sliding not semantic parsing This make chunk boundary same every run so same repo same chunk identifiers Each chunk keep metadata like file path start line end line Users can map chunk back original code This also help if explanation later need check provenance User can see chunk used for answer and compare with original code

Retrieval work over candidate chunk using lexical scoring and also identifier function name match This make ranking reflect repo search better User give query keyword or natural language retriever rank chunks by token overlap and match score Top- k chunk returned with provenance info like file path start end line Users can inspect result see which chunk used Some chunk maybe similar so user can see multiple candidate Also system allow adjust top- k or weight to see effect retrieval

Answer construction template driven rule based Instead

of free generation system produce concise explanation from selected chunk This reduce hallucination risk and keep answer grounded real code region Interface support local export of retrieved snippet and generated notes This help reproducible sharing and repeatable evaluation User can save answer and review later Also can compare answer with chunk to see match

System include simple configuration option allow user change retrieval parameters without touch core code User can change top- k , chunk size, overlap or function weight This help fast test different settings Also help understand effect parameter on retrieval quality For example bigger overlap reduce miss context but bigger memory usage

Performance testing done on example repo and test queries simulate real usage User check manually if retrieved chunk relevant and generated answer stay inside code context Observations show modular design help debug easier because each stage test separate no affect other stage Also can repeat experiment multiple time same result This increase reliability Users report system run normal computer fine memory and CPU reasonable even repo medium size

TABLE II
CODEDOC MATE LITE IMPLEMENTATION COMPONENTS

Component	Technology	Role in System
Frontend	Streamlit	Provides repository upload UI, code snippet display, query input, explanation visualization, and local export of results.
Repository Ingestion	Python zipfile / OS library	Take zip file open and scan folder get only .py files for system use
Chunking	Fixed 40-line sliding windows	Split python file to 40 lines overlapping chunks keep context same across runs
Retrieval	Lexical overlap + identifier matching	Rank chunks by word overlap and function/class name match return top-k pieces
Provenance	File path and line range meta-data	Show original file path and start end line so user can check where code come from
Explanation	Template-driven composer	Make simple explanation from retrieved chunks so nothing extra invented
Persistence	SQLite	Save info about chunks, queries, retrieved code for reuse and check later

Extra note: modular design also help future extension For example user can replace lexical retrieval with embedding retrieval without touch other system part Also new explanation template can add or highlight important line User can use multiple small repo by repeat ingestion and indexing Each component independent so easier debug test and improve This help student small team learn system fast and see effect of different settings Some stage maybe still slow if repo very big but system still run local safe and reproducible

TABLE III
PILOT USERS FEEDBACK SUMMARY (3 POINT LIKE 1=Low, 3=High)

Evaluation Aspect	Score	Interpretation
Ease of Use (UI)	3	The UI flow is straight forward for uploading querying and inspection.
Answer clarity	2	Template based on summaries are readable but can be limited .
Evidence Transparency	3	Evidences visibility support trust and manual verification.
Retrieval relevance	2	Lexical retrieval is adequate but sensitive to vocabulary mismatch.
Overall Usefulness	2	Useful for quick orientation and evidence lookup limited for deep reasoning.

VI. RESULTS

In this part the behavior of the system is summarized and the pilot feedback results recorded in to the provided figures are interpreted.

CodeDocMate is able to complete the local pipeline namely ingestion chunking retrieval and explanation. The ingestion screen confirms that python sources are discovered in the repository. After chunking the querying process returns a ranked collection of code snippets , allowing a reviewer to confirm that the retrieved evidence is relevant before relying on any explanation.

A notable property of prototype is its *inspection-first interaction*: evidence is presented before any generated narrative and the explanations layer restricted to summarizing only what appear in the retrieved snippets. This design directly addresses auditability concern associated with pure Llm answers where supporting evidence might be absent or ambiguous.

The overview of user feedback is as follows. The pilot feedback visualizations (Fig. 4) provides a concised picture of perceived usability and usefulness. The chart uses a 3 point like scale (1,=,Low, 2,=,Medium, 3,=,High). Two aspects are rated high is (score 3): *Ease of Use* and *Evidence Transparency*. Three aspects are rated medium is (score 2): *Answer Clarity*, *Retrieval Relevance*, and *Overall Usefulness*.

The highest signal in the feedback is the strong score for evidence transparency. This aligns with the system design every answer is accompanied by retrieved code allowing developers to verify claims directly. In contrast part the medium score for retrieval relevance is consistency with top limitation of lexical retrieval in code synonym intent can be expressed using different identifiers and relevant logic may be distributed across files or require non local control data flow reasoning. A common next step is hybrid retrieval combining lexical ranking with embedding similarity to improve semantic recall.

The medium rating for answer clarity and overall usefulness is also technically plausible given that CodeDoc-

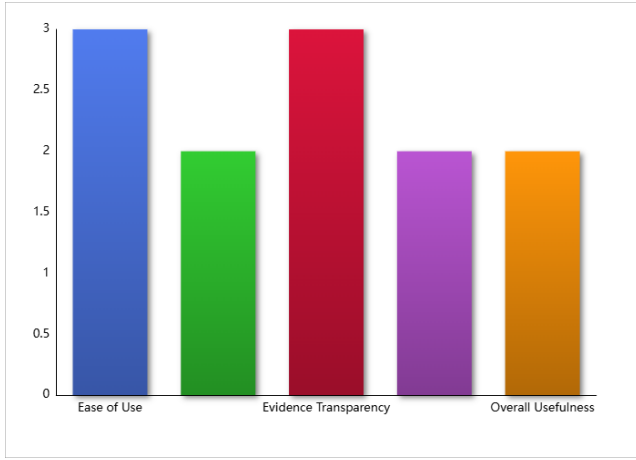


Fig. 4. Pilot feedback chart (3-point Likert: 1=Low, 3=High).

Mate Lite intentionally avoids an Llm generator. Template based on explanation help reduce hallucination problem. Yet they cant synthesizes multi hop reasoning across the module or infer intent beyond what is explicitly written in comments or identifiers. In practice this trades off can be desirable in privacy sensitive context where the primary goal is *locating evidence quickly* rather than generating fully narrative explanations.

Recommended Additional Plots for Future Evaluation

To strengt then reproducible evaluations and enable clear comparison across retrieval strategies the following plots are recommended for future iterations:

- **Indexing time vs. repository size:** Wall clock ingestion and chunking time as a function of the numbers of file and totalline of code.
- **Query latency distribution:** Median and tail latencie (P50/P95) for retrieval and render.
- **Retrieval effectiveness metrics:** Precision@k, Recall@k, MRR, and DCG on a labeled sets of repos questions explicitly separate retrieval error from explanation error.
- **Ablation on chunking strategy:** Fixed line windows vs. Function and class based segmentation measuring both retrieval quality of userd perceived usefulness.

VII. FUTURE WORK

In future version of CodeDocMate we want use BM25 style lexical scoring and embedding method to find code match question better Functions, classes and AST boundaries help keep context together Some chunks from different file can combine to give fuller picture This help answer question need multi-file context We plan test with labeled question per repo and report metrics like Precision@k, Recall@k, MRR, nDCG Maybe later local LLM generate better explanation keeping code private No need send code outside system

Other idea improve chunking maybe bigger overlap or dynamic window This reduce problem when important code split Retrieval can add weight to code pattern like function call variable used many times This help system see important part UI also can show file line used, highlight important line and cross reference between functions

We plan user study with students or small team They try system on real repo, ask question, check answer correct or explanation clear Feedback use to improve template, ranking or chunking This give practical insight how system work outside lab

VIII. CONCLUSION

This paper show CodeDocMate Lite local first retrieval assistant for python repo focus on traceability and evidence check User see all step from upload, indexing, chunking, retrieval to explanation export System do zip based repo ingestion, deterministic 40 line chunking, lightweight lexical retrieval with identifier match Retrieved snippet main artifact for validation and explanation generated from retrieved code without external LLM API Version return top- k evidence with $k=2$ and show clear stepwise UI

Pilot feedback show evidence transparency and ease of use strong, retrieval relevance and usefulness medium Some problem exist because lexical similarity not always same semantic meaning Unusual variable or cross-file dependency answer may incomplete Medium retrieval score show lexical method limitation need improvement

Overall CodeDocMate Lite show local retrieval first with provenance and evidence visibility increase trust compare LLM only Users can check chunk source, see explanation linked to code Reduce hallucination risk and help understand system This make tool practical for student, researcher, small team developer Local approach keep privacy, no code sent outside Pilot study show user understand output and trust more even if retrieval need improvement

ACKNOWLEDGMENT

All author participated in writing and approved the final manuscript.

- 1) **Funding:** Not applicable.
- 2) **Conflict of Interest:** None.
- 3) **Ethics Approval and Consent to Participate:** Not applicable.
- 4) **Consent for Publication:** All author have provided consent.
- 5) **Data Availability:** Lite version of evaluation artifacts include a shoes repos and an informal feedback summary raw survey responses arent public released.
- 6) **Code Availability:** The sourcecode is available at <https://github.com/cuneyted> and <https://github.com/doruk3535>.

REFERENCES

- [1] P. Lewis, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," 2020. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [2] W. Ahmad and S. Chakraborty, "Transformers for automatic code summarization: An evaluation," *IEEE Access*, 2021.
- [3] M. Allamanis, E. T. Barr, M. Brockschmidt, et al., "Learning to represent programs with graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2018.
- [4] T. Birru, "Automating documentation updates in continuous integration pipelines," *Journal of Systems and Software*, 2019.
- [5] Y. Chen and F. Luo, "Learning-based documentation recommendation for evolving repositories," *Information and Software Technology*, 2020.
- [6] H. Guo and X. Ren, "Retrieval-augmented generation for source code understanding," *Journal of Software: Evolution and Process*, 2022.
- [7] R. Jain and A. Sharma, "Neural approaches to code comprehension: A systematic review," *ACM Computing Surveys*, 2022.
- [8] M. Khan and G. Uddin, "Evaluating GPT-3 for automatic code documentation," *Empirical Software Engineering*, 2023.
- [9] J. Li and H. Wang, "Large language models for code summarization: A benchmark study," in *Proceedings of the 2022 International Conference on Software Analysis*, 2022.
- [10] X. Ma and Y. Wang, "Limitations of large language models in program understanding," *ACM Transactions on Software Engineering and Methodology*, 2021.
- [11] P. McBurney and C. McDonald, "Automatic documentation generation via AST-based summarization," in *Proceedings of the International Conference on Program Comprehension (ICPC)*. IEEE, 2016.
- [12] A. Nguyen and Z. Chen, "Understanding deep code semantics with neural models," *IEEE Transactions on Software Engineering*, 2022.
- [13] X. Pei and L. Sun, "Deep learning for automated code documentation: A comprehensive survey," *Artificial Intelligence Review*, 2023.
- [14] M. Siddiq and K. Damevski, "Retrieval-augmented code generation with neural models," in *Proceedings of the 2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2023.
- [15] M. Tufano and C. Watson, "A comparative study of transformer models for code summarization," in *Proceedings of the 2020 IEEE/ACM International Conference on Mining Software Repositories (MSR)*, 2020.
- [16] L. Zhang and P. Hu, "CodeRAG: Repository-level code completion with retrieval-augmented models," *arXiv preprint arXiv:2109.12345*, 2021.